

Just KNN

June Lemieux

2024-12-13

libraries/etc

```
library(tidymodels)
```

```
## -- Attaching packages ----- tidymodels 1.2.0 --

## v broom      1.0.7    v recipes      1.1.0
## v dials      1.3.0    v rsample      1.2.1
## v dplyr      1.1.4    v tibble       3.2.1
## v ggplot2    3.5.1    v tidyr        1.3.1
## v infer      1.0.7    v tune         1.2.1
## v modeldata  1.4.0    v workflows    1.1.4
## v parsnip     1.2.1    v workflowsets 1.1.0
## v purrr      1.0.2    v yardstick    1.3.1

## -- Conflicts ----- tidymodels_conflicts() --
## x purrr::discard() masks scales::discard()
## x dplyr::filter()   masks stats::filter()
## x dplyr::lag()      masks stats::lag()
## x recipes::step()   masks stats::step()
## * Use suppressPackageStartupMessages() to eliminate package startup messages
```

```
library(ggplot2)
library(vip)
```

```
## Warning: package 'vip' was built under R version 4.4.2

##
## Attaching package: 'vip'

## The following object is masked from 'package:utils':
##
##      vi
```

```
library(ROCR)
```

```
## Warning: package 'ROCR' was built under R version 4.4.2
```

```
options(scipen=4)
```

```
## Data Processing
# Initial data import
customer_data <- read.csv("~/00 Merrimack/DSE6111/Course Project/customer_data_1.csv", header = TRUE)

customer_data$Attrition_Flag <- ifelse(customer_data$Attrition_Flag == "Attrited Customer", 1, 0)
customer_data$Attrition_Flag <- factor(customer_data$Attrition_Flag)

# check levels
levels(customer_data$Attrition_Flag)
```

```
## [1] "0" "1"
```

```
# set seed
set.seed(10)

# get rid of ClientNum identifier
customer_data <- customer_data[ -c(1) ]

# create data split
data_split <- initial_split(customer_data, strata = "Attrition_Flag", prop = 0.75)

# create training and test sets
training_set <- training(data_split)
test_set <- testing(data_split)

# row count per set
nrow(training_set)
```

```
## [1] 7595
```

```
nrow(test_set)
```

```
## [1] 2532
```

```
# use cv folds and tune
cv_set <- vfold_cv(training_set, v = 10)
```

```
# create recipe for preprocessing - only numeric var at first
attrition_recipe_knn <-
  recipe(
    Attrition_Flag ~ Months_on_book + Total_Relationship_Count + Months_Inactive_12_mon + Contacts_Count +
      Total_Amt_Chng_Q4_Q1 + Total_Trans_Amt + Total_Trans_Ct + Total_Ct_Chng_Q4_Q1 +
      Avg_Utilization_Ratio,
    data = training_set
  ) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())
```

```

# model with KNN
knn_spec <- nearest_neighbor(neighbors = tune()) %>%
  set_engine("kknn") %>%
  set_mode("classification") # need to double check this setting

# create workflow
knn_workflow <- workflow() %>%
  add_recipe(attrition_recipe_knn) %>%
  add_model(knn_spec)

# build tuning grid
knn_tune_grid <- tibble(neighbors = c(1, 5, 10, 15, 20))

knn_tune <-
  knn_workflow %>%
  tune_grid(
    cv_set,
    grid = knn_tune_grid,
    control = control_grid(save_pred = TRUE),
    metrics = metric_set(roc_auc, accuracy)
  )

```

```
## Warning: package 'kknn' was built under R version 4.4.2
```

```

knn_rs_metrics <- knn_tune %>% collect_metrics()
knn_rs_predictions <- knn_tune %>% collect_predictions()

knn_rs_metrics

```

```
## # A tibble: 10 x 7
```

	neighbors	.metric	.estimator	mean	n	std_err	.config
	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
## 1	1	accuracy	binary	0.899	10	0.00298	Preprocessor1_Model11
## 2	1	roc_auc	binary	0.804	10	0.00536	Preprocessor1_Model11
## 3	5	accuracy	binary	0.913	10	0.00187	Preprocessor1_Model12
## 4	5	roc_auc	binary	0.920	10	0.00357	Preprocessor1_Model12
## 5	10	accuracy	binary	0.917	10	0.00207	Preprocessor1_Model13
## 6	10	roc_auc	binary	0.943	10	0.00414	Preprocessor1_Model13
## 7	15	accuracy	binary	0.918	10	0.00272	Preprocessor1_Model14
## 8	15	roc_auc	binary	0.948	10	0.00394	Preprocessor1_Model14
## 9	20	accuracy	binary	0.918	10	0.00273	Preprocessor1_Model15
## 10	20	roc_auc	binary	0.953	10	0.00330	Preprocessor1_Model15

```

# roc_auc and accuracy
knn_tune %>% show_best(metric = "roc_auc")

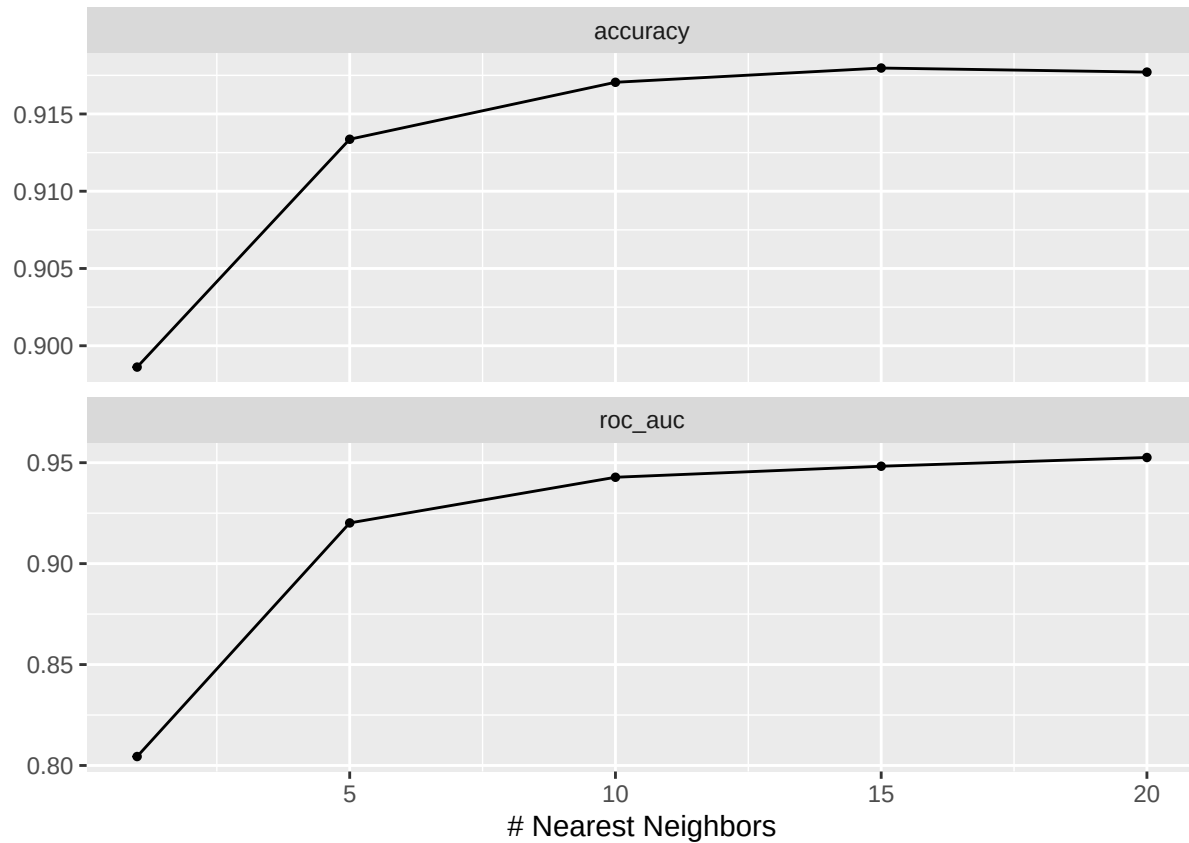
```

```
## # A tibble: 5 x 7
```

	neighbors	.metric	.estimator	mean	n	std_err	.config
	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
## 1	20	roc_auc	binary	0.953	10	0.00330	Preprocessor1_Model15
## 2	15	roc_auc	binary	0.948	10	0.00394	Preprocessor1_Model14
## 3	10	roc_auc	binary	0.943	10	0.00414	Preprocessor1_Model13

```
## 4      5 roc_auc binary    0.920    10 0.00357 Preprocessor1_Model2
## 5      1 roc_auc binary    0.804    10 0.00536 Preprocessor1_Model1
```

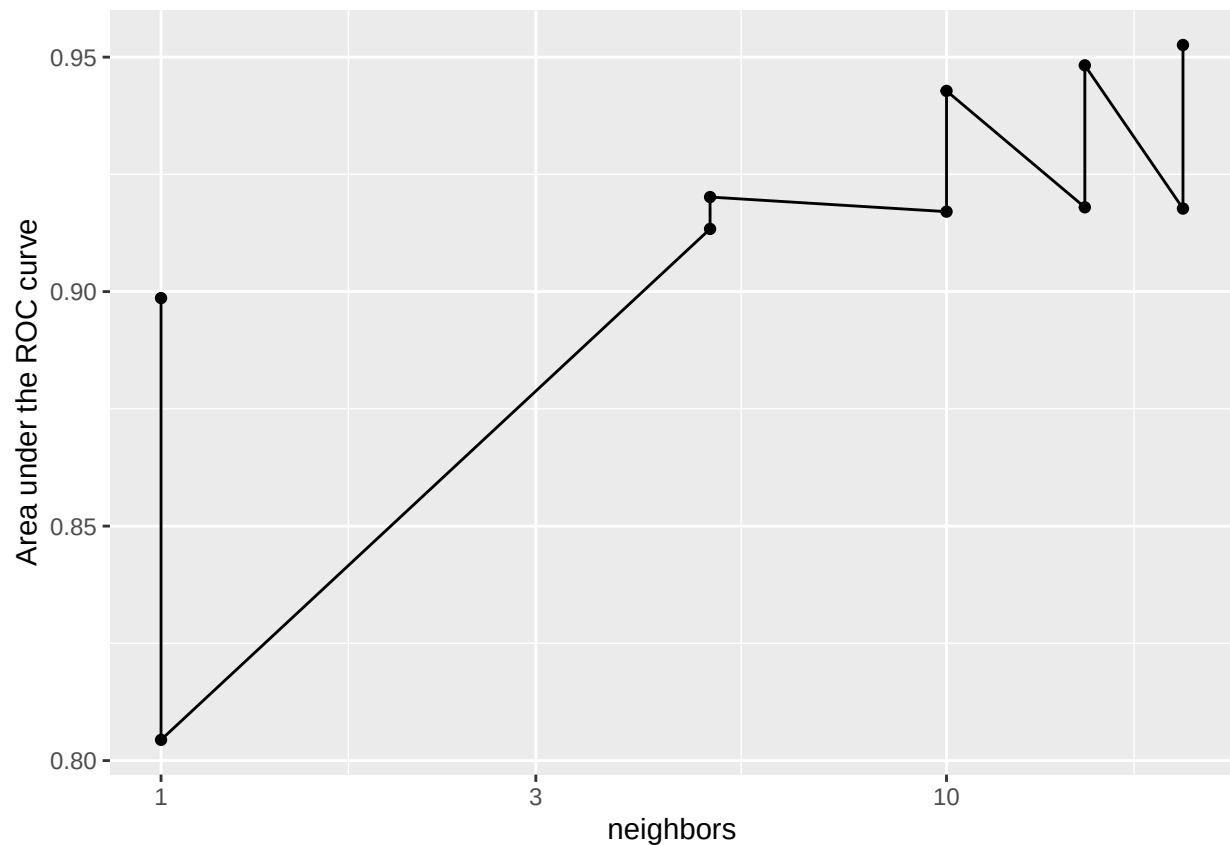
```
autoplot(knn_tune)
```



```
best_auc <- knn_tune %>% select_best(metric = "roc_auc")
best_acc <- knn_tune %>% select_best(metric = "accuracy")
best_neighbors <- knn_tune %>% select_best(metric = 'roc_auc')
```

```
# plot
knn_plot <-
  knn_tune %>%
  collect_metrics() %>%
  ggplot(aes(x = neighbors, y = mean)) +
  geom_point() +
  geom_line() +
  ylab("Area under the ROC curve") +
  scale_x_log10(labels = scales::label_number())
```

```
knn_plot
```



```
final_knn_workflow <- finalize_workflow(
  knn_workflow,
  best_neighbors
)

last_knn_fit <-
  final_knn_workflow %>%
  last_fit(data_split)

knn_metrics <- last_knn_fit %>% collect_metrics()
```

```
# feature importance - not for KNN as is
# last_knn_fit %>%
# extract_fit_parsnip() %>%
# vip(num_features = 10)
```

#If you are set on using KNN though, then the best way to estimate feature importance is by taking the

```
# compare to the null model
null_classification <- null_model() %>%
  set_engine("parsnip") %>%
  set_mode("classification")

null_knn <- workflow() %>%
  add_recipe(attrition_recipe_knn) %>%
  add_model(null_classification) %>%
```

```

fit_resamples(
  cv_set
)

null_metrics <- null_knn %>% collect_metrics()

##### plot ROC curve #####
test_results <-
  test_set %>%
  select(Attrition_Flag) %>%
  bind_cols (
    last_knn_fit %>% collect_predictions() %>%
    select(p_1 = .pred_1)
  )

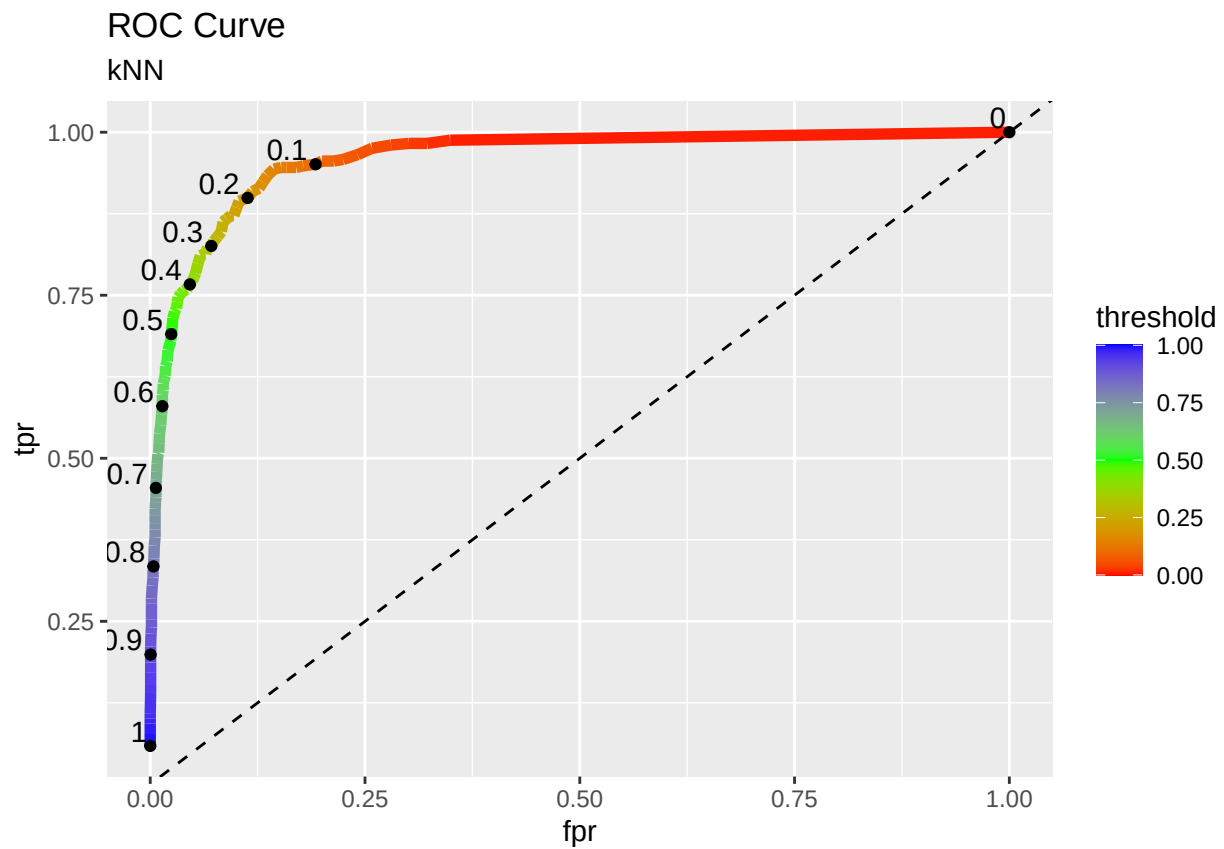
roc_data <- data.frame(threshold = seq(1, 0, -0.01), fpr = 0, tpr = 0)
for (i in roc_data$threshold) {
  over_threshold <- test_results[test_results$p_1 >= i, ]

  fpr <- sum(over_threshold$Attrition_Flag == 0)/sum(test_results$Attrition_Flag==0)
  roc_data[roc_data$threshold==i, "fpr"] <- fpr

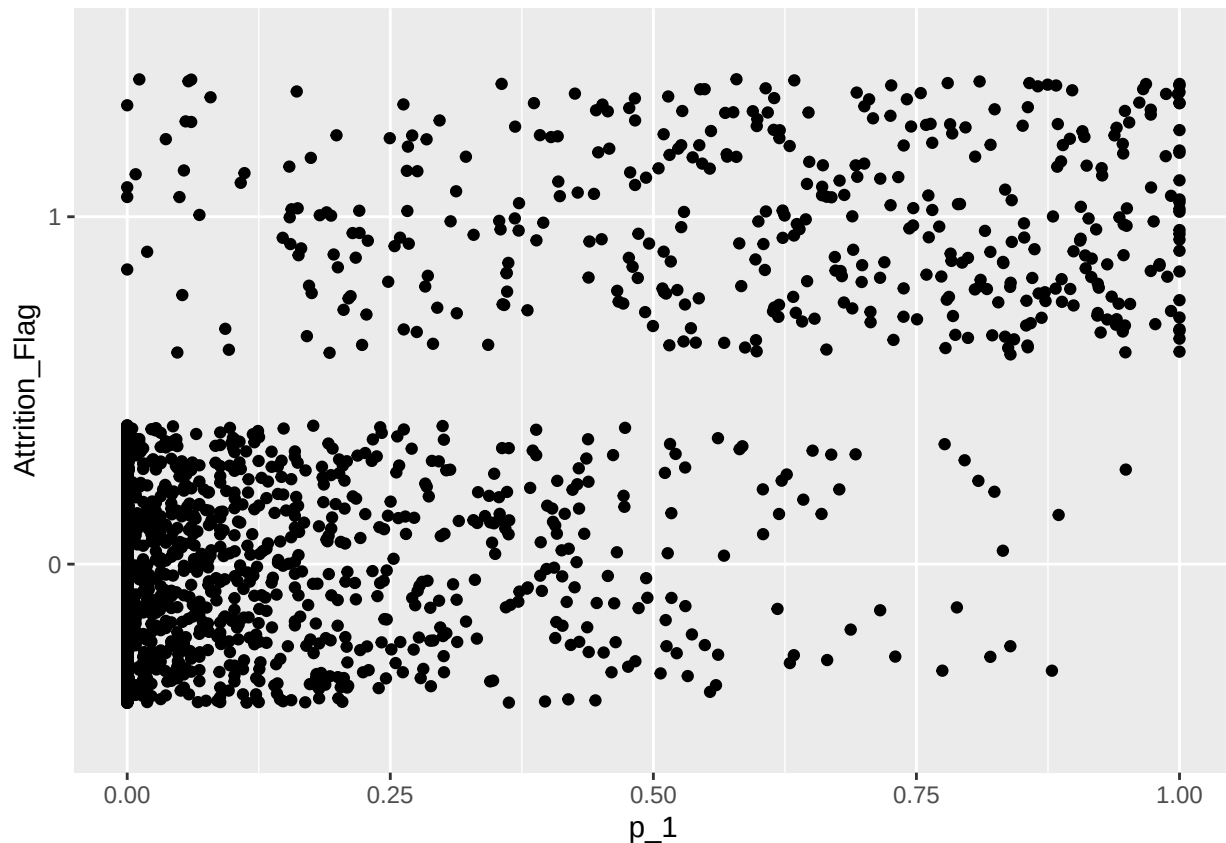
  tpr <- sum(over_threshold$Attrition_Flag == 1)/sum(test_results$Attrition_Flag==1)
  roc_data[roc_data$threshold==i, "tpr"] <- tpr
}

ggplot() +
  geom_line(data = roc_data, aes(x = fpr, y = tpr, color = threshold), linewidth = 2) +
  scale_color_gradientn(colors = rainbow(3)) +
  geom_abline(intercept = 0, slope = 1, lty = 2) +
  geom_point(data = roc_data[seq(1, 101, 10), ], aes(x = fpr, y = tpr)) +
  geom_text(data = roc_data[seq(1, 101, 10), ],
    aes(x = fpr, y = tpr, label = threshold, hjust = 1.2, vjust = -0.2)) +
  ggtitle("ROC Curve", subtitle = "kNN")

```



```
##### ROC curve calculation breakdown #####
ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +
  geom_jitter()
```



```
threshold <- 0.45

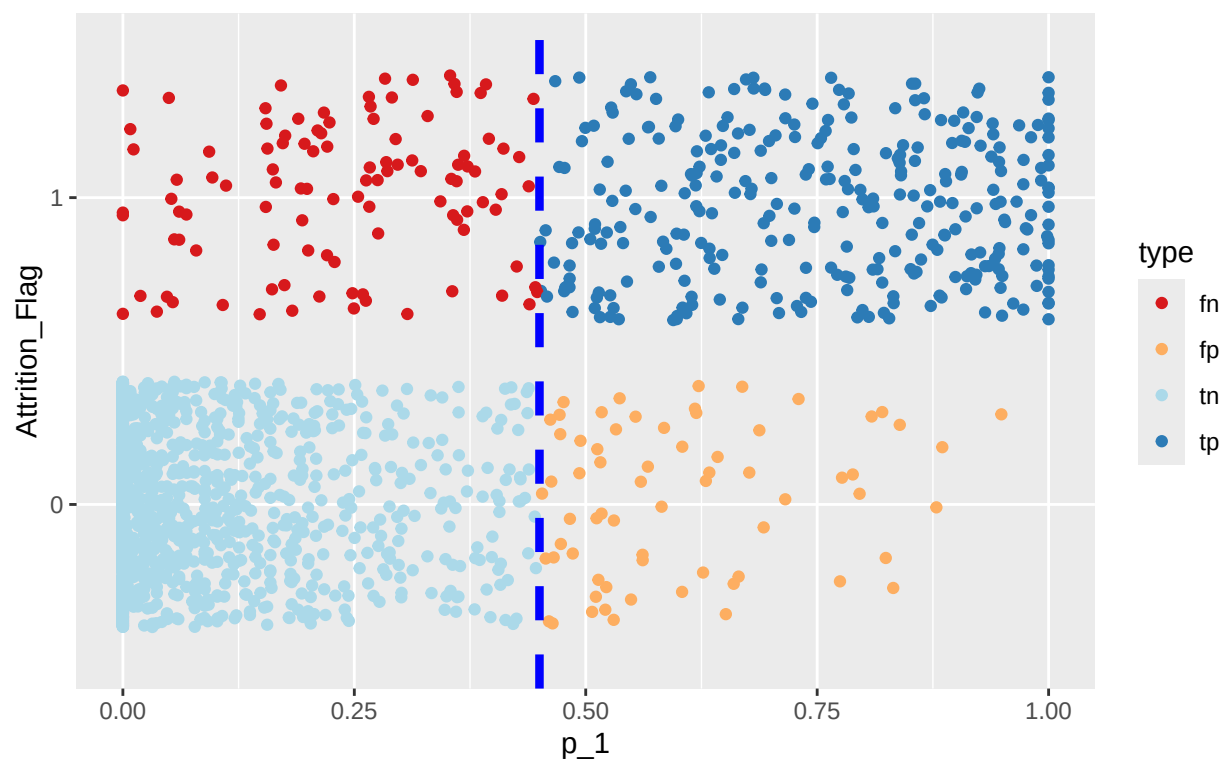
test_results$predictions <- ifelse(test_results$p_1 >= threshold, 1, 0)
tp <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, ])
fp <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, ])
tn <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, ])
fn <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, ])

test_results$type <- ""
test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, "type"] <- "tp"
test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, "type"] <- "fp"
test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, "type"] <- "tn"
test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, "type"] <- "fn"

ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +
  geom_jitter(aes(colour = type)) +
  geom_vline(xintercept = threshold, linetype = "dashed", color = "blue", linewidth = 1.5) +
  scale_color_brewer(palette = "RdYlBu") +
  ggtitle("ROC Curve Calculation Breakdown", subtitle = "kNN")
```


ROC Curve Calculation Breakdown

kNN



```
fpr <- fp/(fp + tn)
tpr <- tp/(tp + fn)
```

```
# confusion matrix data
tn
```

```
## [1] 2058
```

```
fn
```

```
## [1] 106
```

```
fp
```

```
## [1] 67
```

```
tp
```

```
## [1] 301
```